By designating `d.friends` for the cy position, we get circles placed with their depth based on the value of the `friends` attribute. Circles placed lower in the chart represent people in our dataset who have more friends. Circles are arranged from left to right using the old array-position trick you learned earlier in chapter 2. In figure 4.3, you can see that it's not much of a scatterplot.

Next, we need to build scales to make this fit better on our SVG canvas:

```
var xExtent = d3.extent(scatterData, function(d) {
                return d.salary;
              });
var yExtent = d3.extent(scatterData, function(d) {
                return d.friends;
              });
var xScale = d3.scale.linear().domain(xExtent).range([0,500]);
var yScale = d3.scale.linear().domain(yExtent).range([0,500]);
d3.select("svg").selectAll("circle")
          .data(scatterData).enter().append("circle")
          .attr("r", 5).attr("cx", function(d) {
                return xScale(d.salary);
          }).attr("cy", function(d) {
                return yScale(d.friends);
          });
```

The result, in figure 4.4, is a true scatterplot, with points representing people arranged by number of friends along the y-axis and amount of salary along the x-axis.

This chart, like most charts, is practically useless without a way of expressing to the reader what the position of the elements means. One way of accomplishing this is using well-formatted axis labels. Although we could use the same method for binding data and appending elements to create lines and ticks (which are just lines representing equidistant points along an axis) and labels for an axis, D3 provides `d3.svg.axis()`, which we can use to create these elements based on the scales we used to display the data. After we create an axis function, we define how we want our axis to appear. Then

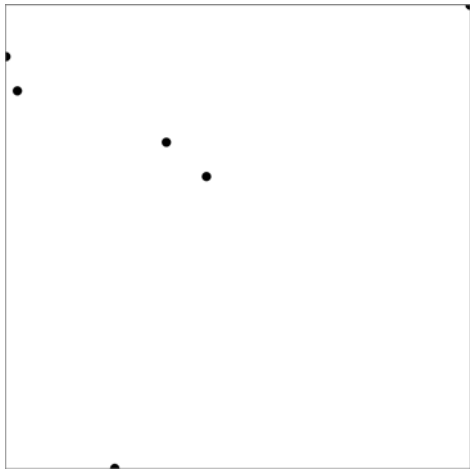


Figure 4.4 Any point closer to the bottom has more friends, and any point closer to the right has a higher salary. But that's not clear at all without labels, which we're going to make.